Administrator Guide

Artifact Publishing and Security Center

Nexus Portal 1.0.0 | Production | PostgreSQL-backed

A complete operational reference for the two portal modules that control artifact intake and registry security. This guide explains every section, field, option, button, status, and action in clear administrator-focused language.

Note: This document is aligned with the production portal UI and behavior. It is written in English and replaces the previous Persian PDF guide.

Generated: 2026-07-16 20:09

Table of Contents

1. Purpose and Operating Assumptions

Artifact Publishing and Security Center are the two modules responsible for controlled artifact intake and controlled registry access. Artifact Publishing imports approved software into Nexus. Security Center defines who and what networks may access Nexus registry services.

- The portal version is 1.0.0 and uses PostgreSQL for portal state.
- Nexus must be healthy and reachable from the portal container.
- Docker publishing requires Docker socket access from the portal container.
- Package fetch workflows require access to Docker Hub, PyPI, Debian/Ubuntu mirrors, or approved internal mirrors.
- Security Center changes are audited and explicit, but some changes require Sync Nexus Access or Apply IP Firewall before they affect Nexus or the host firewall.

```
Health check example:
curl -fsS http://localhost:8095/health
Expected: {"status":"ok","version":"1.0.0","database":"postgresql"}
```

2. Artifact Publishing Module

Artifact Publishing is used to import approved Docker images, Docker image archives, Python packages, and Debian/Ubuntu packages into Nexus repositories. Each operation creates a publish job with status, logs, source, target, repository, checksum where available, and error details.

Area	Purpose	Primary Output
Summary cards	Show publish job counts by state.	Operational visibility.
Container Image by Name	Pull or reuse a named image and push it to Nexus.	Docker image stored in docker-hosted.
Container Image Archive	Upload a docker save archive and publish it.	Internal image stored in Nexus.
Python Package Fetch	Fetch Python packages and optional dependencies.	Artifacts in PyPI hosted repository.
Debian Package Fetch	Fetch .deb packages for a selected release.	Artifacts in APT hosted repository.
Publishing Logs	Display detailed logs for the selected publish job.	Actionable command output and errors.
Publish History	Show past publish jobs and statuses.	Audit-friendly publishing record.

2.1 Summary Cards

Card	Meaning	How to Use It
Publish Jobs	Total number of publish jobs currently listed.	Use it to understand publishing activity volume.
Running	Jobs in QUEUED or RUNNING state.	If this stays non-zero for too long, inspect logs and portal container health.
Success	Jobs completed successfully.	Validate artifacts in Nexus after success.
Failed	Jobs that ended with an error.	Open View Logs to identify root cause.

2.2 Container Image by Name

Use this workflow when you know the Docker image name and tag. The portal checks/pulls the source image, tags it for the Nexus hosted registry, logs in to Nexus, and pushes the image.

Field / Action	Expected Value	When and Why to Use It	Expected Result
Source image	A Docker image reference with tag, for example nginx:1.27 or alpine:3.20.	Use the exact approved source image. Avoid latest because it is not reproducible.	The portal pulls or reuses the image.
Target image path	A destination name without registry host, for example approved/nginx:1.27. May be blank.	Use this to rename images into internal namespaces. Blank means same name as source.	The image is tagged for Nexus.
Repository	Usually docker-hosted.	Select the Nexus Docker hosted repository that should store the image.	The image is pushed to the repository.
Publish Image	Button.	Starts the Docker publishing job.	A job ID appears and logs become available.

Publish result	Read-only feedback text.	Confirms job creation or displays validation/API errors.	Shows job ID or FAILED message.

```
Example:
Source image: alpine:3.20
Target image path: approved/alpine:3.20
Repository: docker-hosted
```

Important: Production recommendation: require pinned image tags and block latest tags. This preserves repeatability and improves auditability.

2.3 Container Image Archive

Use this workflow for internally built Docker images exported with docker save. This is the best option when the image comes from an internal build pipeline or should not be pulled from Docker Hub.

Field / Action	Expected Value	When and Why to Use It	Expected Result
Target image	Internal destination name such as internal/myapp:1.0.0.	Required when the archive contains only an image ID or when you want a controlled internal name.	The loaded image is tagged with this name.
Repository	Usually docker-hosted.	Defines the Nexus repository that receives the image.	Image is pushed to the selected hosted registry.
File	.tar, .tar.gz, or .tgz archive produced by docker save.	Use for locally built/exported images.	Portal uploads and processes the archive.
Upload Archive	Button.	Starts archive upload and publishing.	A publish job is created and logged.
Archive result	Read-only feedback text.	Shows upload/job creation state or errors.	Displays job ID or FAILED message.

```
Create an archive:
docker build -t internal/myapp:1.0.0 .
docker save -o /tmp/myapp-1.0.0.tar internal/myapp:1.0.0
```

Note: Large archives may take time to upload and process. Keep the browser open until the job is created; then monitor progress through Publish History and View Logs.

2.4 Python Package Fetch

Use this workflow to fetch Python packages by name/version and publish them into a Nexus PyPI hosted repository. It is designed for preparing offline or controlled Python dependency sets.

Field / Option	Expected Value	When and Why to Use It	Expected Result
Package name	A valid package name such as requests or click.	Identifies the Python package to fetch.	pip download is executed for the package.
Version	Exact version such as 2.32.4, or blank.	Use exact versions for production. Blank means latest allowed at fetch time.	Selected package files are downloaded.
Runtime	Python 3 or Python 2 legacy.	Choose Python 3 for modern packages. Use Python 2 only for legacy systems.	The portal uses the matching runtime workflow.
Repository	Usually pypi-hosted or pypi2-hosted.	Determines the Nexus PyPI hosted repository.	Packages are uploaded to Nexus.
Include dependencies	Checkbox. Usually enabled for offline use.	Fetches transitive dependencies with the package.	Dependency artifacts are also published.
Fetch & Publish	Button.	Starts the package fetch job.	Job appears in Publish History.
Result message	Read-only feedback text.	Shows job creation or validation errors.	Displays job ID or FAILED message.

Important: Leaving Version blank can produce different results over time. For production dependency sets, pin package versions and record approvals.

2.5 Debian Package Fetch

Use this workflow to fetch Debian/Ubuntu .deb packages for a target operating system release and publish them to a Nexus APT hosted repository.

Field / Option	Expected Value	When and Why to Use It	Expected Result
Package name	APT package name such as curl, vim, or net-tools.	Identifies the Debian/Ubuntu package to fetch.	APT download workflow starts.
Version	Exact package version or blank.	Use exact version when you need deterministic releases. Blank uses the release default.	.deb files for the selected version are fetched.
Target release	Ubuntu 24.04 Noble, Ubuntu 25.04 Plucky, Ubuntu 26.04 Resolute, Debian 11 Bullseye, Debian 12 Bookworm, or Debian 13 Trixie.	Must match the client systems that will consume the package.	The portal uses repositories for that release.
Repository	Usually apt-internal-hosted.	Defines the Nexus APT hosted repository.	Fetched packages are published to Nexus.
Include dependencies	Checkbox. Usually enabled for offline use.	Downloads dependencies needed by the selected package.	Main package and dependencies are stored.
Fetch & Publish	Button.	Starts the Debian fetch job.	Job appears in Publish History.
Result message	Read-only feedback text.	Shows job creation or errors.	Displays job ID or FAILED message.

Important: Do not mix packages from one release with systems running a different release unless you have validated compatibility. APT dependency mismatches can break offline installations.

Target Release	Use Case
Ubuntu 24.04 Noble	Clients or servers running Ubuntu 24.04.
Ubuntu 25.04 Plucky	Ubuntu 25.04 environments. Availability depends on upstream mirror lifecycle.
Ubuntu 26.04 Resolute	Ubuntu 26.04 environments.
Debian 11 Bullseye	Debian 11 systems.
Debian 12 Bookworm	Debian 12 systems.
Debian 13 Trixie	Debian 13 systems.

2.6 Publishing Logs and Publish History

Control / Status	Meaning	Expected Administrator Action
Refresh	Reloads publish job counters and history from PostgreSQL.	Use after starting a job or when checking completion.
View Logs	Loads logs for the selected publish job.	Use to inspect progress, command output, and failure details.
QUEUED	Job has been created but has not started.	Wait briefly; if stuck, check portal logs.
RUNNING	Job is executing.	Monitor logs until completion.
SUCCESS	Job completed successfully.	Validate the artifact in Nexus and by client pull/install.
FAILED	Job ended with an error.	Read logs; check network, credentials, repository names, package/image names, and upstream availability.
SHA256	Checksum recorded for uploaded/fetched file where available.	Use for audit, integrity tracking, and change verification.

3. Security Center Module

Security Center contains the Registry Access Control module. It centrally manages Nexus identities, groups, permissions, trusted source networks, and host firewall enforcement for protected Nexus ports. It is intended to prevent anyone who merely knows the Nexus address from using the registry.

3.1 Top-Level Actions and Metrics

Control / Metric	Purpose	Dependencies and Expected Result
Sync Nexus Access	Creates or updates Nexus roles and users from portal policy and disables anonymous Nexus access.	Requires healthy Nexus API and valid NEXUS_ADMIN_PASSWORD. Nexus roles/users are synchronized.
Preview Firewall	Displays the exact firewall policy before applying it.	Safe read-only action. Shows allow rules and final drop behavior.
Apply IP Firewall	Applies host firewall rules for protected Nexus ports.	High-risk action. Current admin IP must be trusted first. Enforcement status updates after success.
Groups	Number of access groups.	Each group maps to a Nexus role.
Trusted Accounts	Number of trusted user/service account records.	Enabled accounts should have at least one group.
Trusted IP Rules	Number of enabled trusted IP/CIDR rules.	Only enabled ranges are included in firewall policy.
Protected Ports	Nexus ports controlled by firewall policy.	Usually 8081, 5000, 5001, and 5002.

Important: Apply IP Firewall can block access to Nexus. Always add and verify your current administrator workstation IP/CIDR before applying firewall rules.

3.2 Groups & Permissions

Groups model duties such as pull-only runtime systems, build agents, or release publishers. During Sync Nexus Access, groups become Nexus roles with repository privileges derived from selected permissions.

Field / Action	Expected Value	When and Why to Use It	Expected Result
Group Name	Clear unique duty name such as Production Pullers.	Use names administrators can recognize later.	Stored in portal and mapped to a Nexus role.
Description	Short explanation of who belongs in the group.	Documents intent for audits and handoffs.	Saved with the group.
Permissions	One or more permission checkboxes.	Defines the least-privilege repository capabilities for group members.	Converted to Nexus privileges during sync.
Save Group	Button.	Creates or updates the group.	Group list refreshes; sync is attempted where applicable.
Clear	Button.	Resets the form and exits edit mode.	Fields and selections are cleared.
Existing Groups	Record list.	Shows current groups, descriptions, permissions, and actions.	Admins can edit/delete records.
Edit	Record action.	Loads the group into the form for modification.	Save updates the existing group.
Delete	Record action.	Removes a group only when it is not assigned to accounts.	Record disappears; Nexus role deletion is attempted.

Permission	What It Grants	Recommended Use
docker_pull	Browse/read Docker repositories.	Runtime hosts, CI pullers, approved readers.
docker_push	Add/edit Docker hosted repository content.	Trusted build/publishing systems only.
pypi_read	Read PyPI repositories.	Python build/install clients.
apt_read	Read APT repositories.	Debian/Ubuntu package clients.
raw_read	Read raw repositories.	Bundle consumers, release readers, backup readers.
raw_write	Write raw repositories.	Release automation or backup upload services only.

Note: Best practice: create small role-based groups and grant only the permissions required for the workflow. Avoid giving publisher permissions to runtime systems.

3.3 Trusted Users & Systems

Trusted accounts are explicit Nexus identities for approved people, services, CI systems, runtime hosts, or package clients. Passwords are sent to Nexus but are not stored in the portal.

Field / Action	Expected Value	When and Why to Use It	Expected Result
Username / Service ID	Stable ID such as ci-prod-puller.	Used by Docker, pip, apt, or automation to authenticate.	Becomes the Nexus user ID.
Account Type	Service account or Human user.	Service account for automation; Human user for named administrators.	Improves audit context and naming.
Display Name	Friendly name such as Production CI Puller.	Helps audits, handoffs, and operations.	Saved and synced to Nexus user fields.
Email	Owner or team mailbox, for example ci-prod-puller@example.local.	Nexus expects an email-like value.	Stored on the Nexus user.
Password	Password value or blank.	On create, blank generates a one-time password. On update, blank leaves the Nexus password unchanged.	Password is applied to Nexus; generated password is displayed once.
Enabled	Checkbox.	Disable accounts without deleting policy history.	Disabled accounts remain in policy but cannot authenticate.
Assigned Groups	One or more groups.	Determines Nexus roles and permissions. Required for enabled accounts.	Roles are assigned on sync.
Save Account	Button.	Creates or updates a trusted account.	Record is saved and Nexus sync is attempted.
Clear	Button.	Resets the form and exits edit mode.	Fields and group selections are cleared.
Trusted Accounts	Record list.	Shows current accounts, account type, display metadata, groups, and enabled status.	Admins can edit/delete accounts.
Edit	Record action.	Loads account into the form.	Save updates existing record.
Delete	Record action.	Deletes portal record and attempts to remove Nexus user.	Account disappears after success.

Important: Generated passwords are shown once. Copy them immediately into an approved secret manager. The portal does not store them.

3.4 Trusted IP Addresses & Ranges

Trusted IP rules define which source addresses may reach protected Nexus ports after firewall enforcement. This is network-layer control and complements Nexus authentication and authorization.

Field / Action	Expected Value	When and Why to Use It	Expected Result
Rule Name	Name such as Admin VPN, CI Runners, or Production Nodes.	Identifies ownership and purpose.	Displayed in Trusted IP Rules.
IP / CIDR	Single host like 172.20.102.182/32 or network like 10.20.0.0/24.	Defines approved source clients. Use /32 for one host.	Included as an allow rule when enabled.
Description	Reason or ownership note.	Documents why the source is trusted.	Saved for audit and handoff.
Enabled	Checkbox.	Temporarily include/exclude a rule without deleting it.	Disabled rules are saved but excluded from firewall policy.
Save IP Rule	Button.	Creates or updates a trusted source rule.	Rule list refreshes. Re-apply firewall for enforcement.
Clear	Button.	Resets the IP rule form.	Fields return to create mode.
Trusted IP Rules	Record list.	Shows CIDR, name, description, enabled state, and actions.	Admins can edit/delete rules.
Edit	Record action.	Loads rule into the form.	Save updates the rule.
Delete	Record action.	Deletes the rule from policy.	Re-apply firewall for network effect.

```
Examples:
172.20.102.182/32     One administrator workstation
10.10.20.0/24         CI runner subnet
192.168.50.10/32      One production runtime host
```

3.5 Enforcement Status and Firewall Preview

Area	Purpose	How to Interpret It
Enforcement Status	Shows the last firewall application state.	NEVER_APPLIED means no portal firewall apply has happened. APPLIED means the last apply completed.
Protected ports	Shows controlled Nexus ports.	Typically includes Nexus UI/API and Docker registry connector ports.
Applied at / by	Shows timestamp and actor for last apply.	Used for audit and troubleshooting.
Firewall Preview	Shows generated firewall commands/rules.	Verify allow rules for trusted CIDRs appear before the final drop rule.
Firewall output	Result text after apply.	Use to confirm applied rules or diagnose failures.

Note: A trusted but unauthenticated client should usually receive HTTP 401 from /v2/. That means network access is allowed but credentials are required. An untrusted client should time out or be blocked before reaching Nexus.

4. Practical Use Cases

Use Case	Recommended Configuration
Production runtime pulls approved images	Group: docker_pull only. Account: service account. Trusted IP: runtime host or subnet.
CI imports/publishes internal images	Group: docker_pull + docker_push. Account: CI publisher. Trusted IP: CI runner subnet.
Python build system installs approved packages	Group: pypi_read. Account: build service or user. Trusted IP: build network.
Debian/Ubuntu hosts install packages	Group: apt_read. Account/client if used. Trusted IP: package client subnet.
Release automation stores bundles or backups	Group: raw_read + raw_write. Account: release or backup service. Trusted IP: automation host only.

5. Dependencies, Permissions, and Risks

- Artifact Publishing depends on healthy Nexus repositories and valid Nexus administrator credentials configured in the portal environment.
- Docker workflows depend on the Docker daemon/socket and on Docker registry connectivity to Nexus hosted ports.
- Python and Debian fetch workflows depend on access to external repositories or approved internal mirrors.
- Security Center group/account changes must be synchronized to Nexus before they fully affect registry authorization.
- Trusted IP rule changes must be applied with Apply IP Firewall before they fully affect network access.
- Firewall enforcement is powerful: wrong CIDR values can block legitimate administrators or automation.
- docker_push and raw_write permissions should be treated as high-trust permissions. Grant them only to controlled publisher identities.
- Use pinned image tags and package versions wherever possible to keep builds reproducible and auditable.

6. Troubleshooting Reference

Symptom	Likely Cause	Recommended Check
Publish job is FAILED	Network, repository, credential, image/package name, or upstream availability issue.	Open View Logs; check docker logs airgap-portal; verify Nexus repository names.
Docker reports HTTP response to HTTPS client	Docker daemon does not trust the HTTP registry.	Add registry to insecure-registries and restart Docker.
Pull-only user can push	Wrong group assignment or stale Nexus role sync.	Review assigned groups and run Sync Nexus Access again.
Untrusted client reaches registry	CIDR is too broad or firewall was not applied.	Review Trusted IP Rules, Preview Firewall, and iptables.
Admin is locked out after firewall apply	Admin IP was not trusted or CIDR was wrong.	Use host console access to correct firewall rules.
Python/Debian fetch fails	No network path, wrong package/version, incompatible release, or upstream unavailable.	Check publish logs and test network from portal container.
Sync Nexus Access fails	Nexus unhealthy or wrong admin credential.	Check Nexus status and NEXUS_ADMIN_PASSWORD.

7. Administrator Checklist

- Every published artifact is visible in the correct Nexus repository.
- Each publish job has clear logs and a final status.
- Groups are role-based and least-privilege.
- Every enabled trusted account has at least one group.
- Generated passwords are captured securely outside the portal.
- Nexus anonymous access is disabled after sync.
- Trusted IP rules include only approved hosts or networks.
- Firewall preview is reviewed before apply.
- Untrusted clients are blocked at the network layer after firewall enforcement.
- Audit logs record important actions.
- Backup procedures include PostgreSQL portal data and Nexus data.